

r-valor y l-valor

```
C/C++
#include <iostream>
using namespace std;

void increment(int& x) {
    x = x + 1;
}

int main() {
    int a = 5;
    increment(a);
    cout << a << endl;
}
```

Explicación de este código generada por un asistente de IA

The function `increment` takes `x` as an l-value reference, which means it receives a copy of the original variable `a`. Since `a` is an l-value, it is first converted into an r-value when passed to the function, and then assigned to `x`. Inside the function, modifying `x` only affects this local copy. Therefore, the original variable `a` remains unchanged, and the output will be `5`.

¿Qué errores encontramos en esta explicación?

tipado seguro vs inseguro

¿Cuál de estas dos variantes del mismo código en TypeScript tiene inseguridades de tipos?

Analiza ambos fragmentos y marca cada afirmación como **Verdadero (V)** o **Falso (F)**.

Variante A

TypeScript

```
function processRequest(handler: (data: any) => any, data: any): any {
    return handler(data);
}

function login(data: any): any {
    if (data.username && data.password) {
        return { token: "abc123", role: "user" };
    }
    return null;
}

function pay(data: any): any {
    if (data.token && data.amount) {
        return { status: "ok" };
    }
    return { status: "error" };
}

// Uso
const authResult = processRequest(login, { username: "u", password: "p" });
const paymentResult = processRequest(pay, authResult);
```

Variante B

TypeScript

```
type AuthRequest = {
    username: string;
    password: string;
};

type AuthResponse = {
    token: string;
    role: string;
};

type PaymentRequest = {
```

```

    token: string;
    amount: number;
};

type PaymentResponse = {
    status: "ok" | "error";
};

function login(data: AuthRequest): AuthResponse | null {
    if (data.username && data.password) {
        return { token: "abc123", role: "user" };
    }
    return null;
}

function pay(data: PaymentRequest): PaymentResponse {
    if (data.token && data.amount > 0) {
        return { status: "ok" };
    }
    return { status: "error" };
}

```

1. La Variante A presenta inseguridades de tipos V.
2. El uso de **any** en la Variante A impide que el compilador detecte comportamientos inesperados en funciones. V
3. En la Variante A, **authResult** siempre cumple el contrato esperado por **pay**. V
4. La Variante B define contratos explícitos entre componentes. V
5. En la Variante B, el compilador puede detectar si falta la propiedad **amount**. V
6. La Variante A es más segura porque es más flexible. F
7. En la Variante A, errores de tipo pueden aparecer solo en tiempo de ejecución. V
8. La Variante B elimina completamente cualquier posible error lógico. F
9. En la Variante A, los componentes están acoplados mediante estructuras implícitas no tipadas. V
10. La Variante B mejora la mantenibilidad del código. V

iteración y recursión a la cola

Dados los siguientes códigos:

versión recursiva (no optimizada)

None

```
def factorial(n: Int): Int = {  
  if (n == 0) 1  
  else n * factorial(n - 1)  
}
```

versión recursiva de cola

None

```
def factorialTail(n: Int): Int = {  
  def loop(n: Int, acc: Int): Int = {  
    if (n == 0) acc  
    else loop(n - 1, acc * n)  
  }  
  loop(n, 1)  
}
```

versión iterativa

None

```
def factorialIter(n: Int): Int = {  
  var acc = 1  
  var i = n  
  while (i > 0) {  
    acc *= i  
    i -= 1  
  }  
  acc  
}
```

Encuentre los fallos a la siguiente explicación:

La versión `factorialTail` es equivalente a la iterativa porque el compilador convierte automáticamente cualquier función recursiva en un bucle. Además, al ser recursiva de cola, no usa pila en ningún caso, ya que la optimización de recursión a la cola siempre se aplica. Por eso, `factorial` y `factorialTail` tienen el mismo consumo de memoria, pero la segunda es más rápida porque usa un acumulador.

pasaje por nombre

None

```
def doble(x: => Int): Int = {  
  x + x  
}  
  
var i = 1  
  
val resultado = doble({ i += 1; i })  
println(resultado)
```

Encuentre los fallos a la siguiente explicación:

El parámetro `x` se pasa por nombre, lo que significa que se evalúa una sola vez cuando entra a la función `doble`, y luego ese valor se reutiliza. En este caso, el bloque `{ i += 1; i }` se ejecuta al inicio, incrementando `i` a 2. Luego, `x + x` equivale a `2 + 2`, por lo que el resultado es 4.

Además, como `x` es un parámetro, su valor queda fijado dentro de la función, evitando efectos secundarios adicionales. Por eso, `i` solo se incrementa una vez.